МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №4

"Професійне ООП та система типів Python"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints. Індивідуальний варіант 1: система обліку студентів.

Теоретичні відомості:

Об'єктно-орієнтоване програмування в Python базується на створенні класів, які поєднують внутрішній стан об'єкта та його поведінку. Принцип інкапсуляції забезпечує цілісність даних через приховування деталей реалізації та використання властивостей (@property) для контрольованого доступу до атрибутів. Наслідування дозволяє створювати спеціалізовані підкласи на базі абстрактних базових класів (ABC) із коректним викликом super().__init__(). У сучасній архітектурі перевага надається композиції та інверсії залежностей (DIP), коли зовнішні сервіси передаються через параметри конструктора (Dependency Injection). Структурна типізація через Protocol (PEP 544) із декоратором @runtime_checkable забезпечує гнучкість качиної типізації при збереженні суворої перевірки контракту. Узагальнені класи (Generic, TypeVar) гарантують типобезпеку колекцій та репозиторіїв, а словники TypedDict формалізують структури для обміну даними та звітів. Статичний type checker (mypy) дозволяє виявляти помилки несумісності типів на етапі статичного аналізу коду.

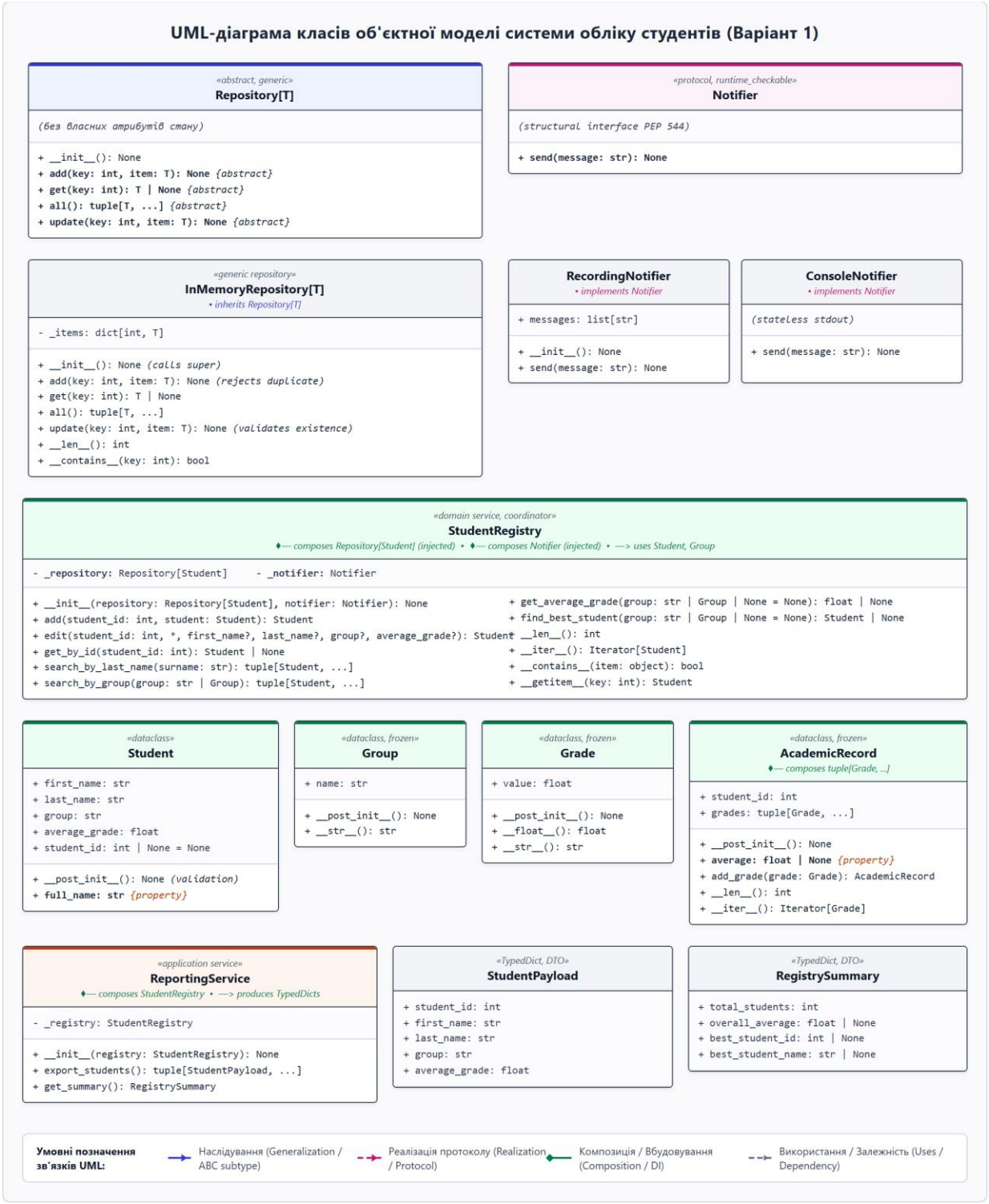
Виконання роботи

1. Визначив завдання варіанта 1: розробити типізовану об'єктну модель системи обліку студентів. Предметна область охоплює облік студентів факультету, їх розподіл по академічних групах, фіксацію оцінок, розрахунок індивідуального та групового середнього бала, пошук за прізвищем та формування підсумкових звітів. Для моделювання предметної області реалізував такі ключові сутності:

- Student: основна сутність студента (ім'я, прізвище, група, середній бал, числовий ідентифікатор);
- Group: незмінний об'єкт значення (Value Object) назви навчальної групи з валідацією;
- Grade: незмінний числовий об'єкт значення оцінки в діапазоні від 0 до 100 балів;
- AcademicRecord: картка успішності, яка містить незмінний кортеж оцінок Grade та розраховує середній бал;
- StudentRegistry: координатор і доменний агрегат із контролем унікальності ідентифікаторів, валідацією та сповіщеннями;
- Repository[T] та InMemoryRepository[T]: абстрактний та конкретний узагальнені репозиторії для збереження сутностей;

- Notifier, ConsoleNotifier та RecordingNotifier: протокол та його реалізації для сповіщення про події в системі;
- ReportingService: сервіс додатку для формування підсумкової статистики та експорту типізованих даних.

2. Спроектував та згенерував аутентичну UML-діаграму класів об'єктної моделі на основі реальних класів проєкту. Діаграма відображає всі реалізовані доменні сутності та чітко розрізняє типи зв'язків:
- наслідування (Generalization): InMemoryRepository[T] наслідує абстрактний базовий клас Repository[T];
- реалізація протоколу (Realization): RecordingNotifier та ConsoleNotifier задовольняють структурний контракт Notifier;
- композиція (Composition): StudentRegistry містить впроваджені залежності Repository[Student] та Notifier, а AcademicRecord містить кортеж об'єктів Grade;
- залежність та використання (Uses): StudentRegistry оперує сутностями Student і Group, а ReportingService використовує StudentRegistry для формування словників StudentPayload та RegistrySummary.



3. Реалізував абстрактний базовий клас Repository[T] із використанням модуля abc та типізації Generic[T] з TypeVar('T'). Використання наслідування тут повністю виправдане, оскільки воно формує єдиний контракт CRUD-операцій для довільних доменних сутностей і гарантує сумісність підтипів за принципом підстановки Лісков (LSP). У класі InMemoryRepository[T] викликаю super().__init__() та перевизначаю всі абстрактні методи.

```python
from abc import ABC, abstractmethod
from typing import Generic, TypeVar

T = TypeVar("T")


class Repository(ABC, Generic[T]):
    # абстрактний базовий клас репозиторію
    def __init__(self) -> None:
        super().__init__()

    @abstractmethod
    def add(self, key: int, item: T) -> None:
        # додавання сутності за ключем
        ...

    @abstractmethod
    def get(self, key: int) -> T | None:
        # отримання сутності за ключем
        ...

    @abstractmethod
    def all(self) -> tuple[T, ...]:
        # повертає всі збережені сутності
        ...

    @abstractmethod
    def update(self, key: int, item: T) -> None:
        # оновлення наявної сутності
        ...
```

Клас InMemoryRepository[T] зберігає сутності у внутрішньому словнику, захищає від дублювання ключів та перевіряє наявність сутності під час оновлення:

```python
class InMemoryRepository(Repository[T]):
    # збереження сутностей у пам'яті
    def __init__(self) -> None:
        super().__init__()
        self._items: dict[int, T] = {}

    def add(self, key: int, item: T) -> None:
        # додаєм новий запис і блокуєм дублікати ключів
        if key in self._items:
            raise ValueError(f"Key {key} already exists")
        self._items[key] = item

    def get(self, key: int) -> T | None:
        # дістаєм запис за ключем
        return self._items.get(key)

    def all(self) -> tuple[T, ...]:
        # незмінний кортеж усіх записів
        return tuple(self._items.values())

    def update(self, key: int, item: T) -> None:
        # оновлюєм існуючий запис
        if key not in self._items:
            raise KeyError(f"Key {key} not found")
        self._items[key] = item

    def __len__(self) -> int:
        return len(self._items)

    def __contains__(self, key: int) -> bool:
        return key in self._items
```

4. Спроектував доменні об'єкти значень Group та Grade як незмінні класи даних (@dataclass із frozen=True та slots=True). У методі __post_init__ перевіряю коректність даних: непорожність назви групи, скінченність та межі оцінки 0-100 балів. Незмінність забезпечує надійність моделі, усуваючи побічні ефекти при передачі оцінок між сервісами.

```python
@dataclass(frozen=True, slots=True)
class Group:
    # валідована назва академічної групи
    name: str

    def __post_init__(self) -> None:
        cleaned = self.name.strip()
        if not cleaned:
            raise ValueError("Group name cannot be empty")
        object.__setattr__(self, "name", cleaned)

    def __str__(self) -> str:
        return self.name


@dataclass(frozen=True, slots=True)
class Grade:
    # скінченна оцінка від 0 до 100 балів
    value: float

    def __post_init__(self) -> None:
        if isinstance(self.value, bool):
            raise TypeError("Grade value cannot be a boolean")
        try:
            val = float(self.value)
        except (TypeError, ValueError) as err:
            raise ValueError("Grade value must be a valid number") from err

        if not math.isfinite(val):
            raise ValueError("Grade must be a finite number")
        if not 0.0 <= val <= 100.0:
            raise ValueError("Grade must be between 0 and 100")
        object.__setattr__(self, "value", val)

    def __float__(self) -> float:
        return self.value

    def __str__(self) -> str:
        return f"{self.value:g}"
```

Для збереження оцінок студента створив клас AcademicRecord, побудований на основі композиції: він містить незмінний кортеж grades: tuple[Grade, ...]. Додавання нової оцінки через метод add_grade повертає новий екземпляр запису через replace, не модифікуючи попередній стан:

```python
@dataclass(frozen=True, slots=True)
class AcademicRecord:
    # академічна картка студента з незмінним списком оцінок
    student_id: int
    grades: tuple[Grade, ...] = field(default_factory=tuple)

    def __post_init__(self) -> None:
        if (
            isinstance(self.student_id, bool)
            or not isinstance(self.student_id, int)
            or self.student_id <= 0
        ):
            raise ValueError("student_id must be a positive integer")

        cleaned_grades: list[Grade] = []
        for item in self.grades:
            if not isinstance(item, Grade):
                raise TypeError("grades must contain only Grade instances")
            cleaned_grades.append(item)
        object.__setattr__(self, "grades", tuple(cleaned_grades))

    @property
    def average(self) -> float | None:
        if not self.grades:
            return None
        return sum(float(g) for g in self.grades) / len(self.grades)

    def add_grade(self, grade: Grade) -> "AcademicRecord":
        if not isinstance(grade, Grade):
            raise TypeError("grade must be a Grade instance")
        return replace(self, grades=(*self.grades, grade))

    def __len__(self) -> int:
        return len(self.grades)

    def __iter__(self) -> Iterator[Grade]:
        return iter(self.grades)
```

5. Реалізував інкапсуляцію та контрольований доступ до атрибутів. У класі Student використав @property full_name для динамічного формування повного імені. Валідація полів студента зосереджена в __post_init__, що унеможливлює створення об'єкта з некоректними даними.

```python
@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float
    student_id: int | None = None

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")
        if self.student_id is not None:
            # перевіряємо шоб айді був цілим додатнім числом і не bool
            if isinstance(self.student_id, bool) or not isinstance(
                self.student_id, int
            ):
                raise TypeError("Student ID must be an integer")
            if self.student_id <= 0:
                raise ValueError("Student ID must be a positive integer")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

У класі AcademicRecord властивість @property average інкапсулює розрахунок середнього бала оцінок або повертає None для порожньої картки. Внутрішній стан репозиторію _items захищений від прямого втручання ззовні. У реєстрі метод edit виконує атомарне оновлення: створення нової копії через replace викликає валідацію в моделі; якщо нові дані некоректні, виникає помилка, а початковий запис у репозиторії залишається цілим:

```
    def edit(
        self,
        student_id: int,
        *,
        first_name: str | None = None,
        last_name: str | None = None,
        group: str | None = None,
        average_grade: float | None = None,
    ) -> Student:
        # атомарне оновлення студента через replace
        existing = self.get_by_id(student_id)
        if existing is None:
            raise KeyError(f"Student with ID {student_id} not found")

        # валідація спрацює в __post_init__, оригінал не пошкодиться при помилці
        updated = replace(
            existing,
            first_name=existing.first_name if first_name is None else first_name,
            last_name=existing.last_name if last_name is None else last_name,
            group=existing.group if group is None else group,
            average_grade=existing.average_grade
            if average_grade is None
            else average_grade,
        )

        self._repository.update(student_id, updated)
        self._notifier.send(f"Updated student {student_id}: {updated.full_name}")
        return updated
```

```
    def get_by_id(self, student_id: int) -> Student | None:
        # пошук за числовим ідентифікатором
        if type(student_id) is not int or student_id <= 0:
            raise ValueError("student_id must be a positive integer")
        return self._repository.get(student_id)

    def search_by_last_name(self, surname: str) -> tuple[Student, ...]:
        # точний пошук за прізвищем без урахування регістру та пробілів
        target = surname.strip().casefold()
        return tuple(
            s
            for s in self._repository.all()
            if s.last_name.strip().casefold() == target
        )

    def search_by_group(self, group: str | Group) -> tuple[Student, ...]:
        # пошук за групою через рядок або об'єкт Group
        group_str = group.name if isinstance(group, Group) else str(group)
        target = group_str.strip().casefold()
        return tuple(
            s for s in self._repository.all() if s.group.strip().casefold() == target
        )
```

```
    def get_average_grade(self, group: str | Group | None = None) -> float | None:
        # обчислення середнього балу для групи або всього реєстру
        students = (
            self.search_by_group(group) if group is not None else self._repository.all()
        )
        if not students:
            return None
        return sum(s.average_grade for s in students) / len(students)

    def find_best_student(self, group: str | Group | None = None) -> Student | None:
        # пошук студента з найвищим середнім балом
        students = (
            self.search_by_group(group) if group is not None else self._repository.all()
        )
        if not students:
            return None
        return max(students, key=lambda s: s.average_grade)
```

Для підтвердження інкапсуляції, контролю унікальності ID та захисту стану при невалідних змінах підготував перевірочний сценарій:

```python
from student_manager.models import Student
from student_manager.repositories import InMemoryRepository
from student_manager.registry import StudentRegistry, ConsoleNotifier


def main() -> None:
    repo = InMemoryRepository[Student]()
    registry = StudentRegistry(repo, ConsoleNotifier())

    # 1. додавання студента з id 1
    s1 = Student("Олена", "Коваль", "ФеП-33", 92.0)
    registry.add(1, s1)

    # 2. спроба додати іншого студента з дублікатом id 1
    s2 = Student("Іван", "Франко", "ФеП-33", 88.0)
    try:
        registry.add(1, s2)
    except ValueError as err:
        print(f"Помилка дублювання ID: {err}")

    # 3. спроба некоректного редагування бала на 120.0
    try:
        registry.edit(1, average_grade=120.0)
    except ValueError as err:
        print(f"Помилка валідації при редагуванні: {err}")

    # 4. перевірка збереження первинного стану студента
    current = registry.get_by_id(1)
    grade = current.average_grade if current else None
    print(f"Стан студента 1 після помилок: бал {grade} (без змін)")
```

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe
.local_workflow/lab4/test_duplicate_and_edit.py
Added student 1: Олена Коваль
Помилка дублювання ID: Key 1 already exists
Помилка валідації при редагуванні: Average grade must be between 0 and 100
Стан студента 1 після помилок: бал 92.0 (без змін)
```

6. Для організації повідомлень створив протокол Notifier через typing.Protocol із декоратором @runtime_checkable. Це дозволяє реалізувати структурну типізацію: будь-який клас із методом send(message: str) -> None вважається коректним нотифікатором і проходить перевірку isinstance(..., Notifier) без явного успадкування. Реалізував ConsoleNotifier для виводу в термінал та RecordingNotifier для збереження повідомлень під час тестування й аудиту:

```python
@runtime_checkable
class Notifier(Protocol):
    # протокол для відправки сповіщень
    def send(self, message: str) -> None: ...


class ConsoleNotifier:
    # вивід сповіщень у стандартну консоль
    def send(self, message: str) -> None:
        print(message)


class RecordingNotifier:
    # запис повідомлень для тестів та аудиту
    def __init__(self) -> None:
        self.messages: list[str] = []

    def send(self, message: str) -> None:
        self.messages.append(message)
```

7. Реалізував спеціальні dunder-методи для надання доменним об'єктам зручного інтерфейсу колекцій Python:
- __len__: визначення кількості студентів у реєстрі len(registry) та оцінок len(record);
- __iter__: підтримка прямої ітерації по студентах у циклах for s in registry;
- __contains__: швидка перевірка наявності студента або його ID оператором in;
- __getitem__: індексація квадратними дужками registry[student_id] за числовим ID.

Також перевантажив __str__ та __float__ для об'єктів Grade, що забезпечує автоматичне приведення оцінки до числа при арифметичних обчисленнях.

```
    def __len__(self) -> int:
        return len(self._repository.all())

    def __iter__(self) -> Iterator[Student]:
        return iter(self._repository.all())

    def __contains__(self, item: object) -> bool:
        # перевірка наявності id або сутності студента
        if isinstance(item, bool):
            return False
        if isinstance(item, int):
            return self._repository.get(item) is not None
        if isinstance(item, Student):
            sid = getattr(item, "student_id", None)
            if sid is not None:
                return self._repository.get(sid) == item
            return item in self._repository.all()
        return False

    def __getitem__(self, key: int) -> Student:
        # доступ за індексом через квадратні дужки
        if isinstance(key, bool) or not isinstance(key, int):
            raise TypeError("Key must be an integer ID")
        student = self._repository.get(key)
        if student is None:
            raise KeyError(f"Student with ID {key} not found")
        return student
```

8. Для обміну даними із зовнішніми системами та формування аналітики описав типізовані словники StudentPayload та RegistrySummary на базі typing.TypedDict. Сервіс ReportingService ізолює завдання генерації звітності від доменного реєстру, надаючи методи експорту записів та формування короткого числового резюме:

```
class StudentPayload(TypedDict):
    # структура даних для експорту та api звітів
    student_id: int
    first_name: str
    last_name: str
    group: str
    average_grade: float


class RegistrySummary(TypedDict):
    # підсумкова статистика реєстру студентів
    total_students: int
    overall_average: float | None
    best_student_id: int | None
    best_student_name: str | None


def student_to_payload(student: Student, student_id: int) -> StudentPayload:
    # перетворює сутність студента у типізований словник
    return {
        "student_id": student_id,
        "first_name": student.first_name,
        "last_name": student.last_name,
        "group": student.group,
        "average_grade": student.average_grade,
    }
```

```python
class ReportingService:
    # сервіс генерації типізованих звітів
    def __init__(self, registry: StudentRegistry) -> None:
        self._registry = registry

    def export_students(self) -> tuple[StudentPayload, ...]:
        # експорт усіх студентів у форматі TypedDict
        result: list[StudentPayload] = []
        for student in self._registry:
            sid = student.student_id
            if sid is None:
                raise ValueError("Registered student must have an ID")
            result.append(student_to_payload(student, sid))
        return tuple(result)

    def get_summary(self) -> RegistrySummary:
        # коротке числове резюме по реєстру
        avg = self._registry.get_average_grade()
        best = self._registry.find_best_student()
        return {
            "total_students": len(self._registry),
            "overall_average": round(avg, 2) if avg is not None else None,
            "best_student_id": getattr(best, "student_id", None) if best else None,
            "best_student_name": best.full_name if best else None,
        }
```

9. Застосував шаблон Dependency Injection (впровадження залежностей через конструктор). Клас StudentRegistry не створює об'єкти репозиторію та нотифікатора самостійно, а отримує їх як аргументи __init__(repository, notifier). Це забезпечує слабку зв'язність і дозволяє вільно комбінувати компоненти.
У структурі проєкту втілено щонайменше три принципи SOLID:
- Single Responsibility Principle (SRP): кожен клас має одну чітку відповідальність. Student і AcademicRecord зберігають та валідують стан, InMemoryRepository займається зберіганням сутностей, StudentRegistry координує бізнес-операції та пошук, а ReportingService відповідає за аналітику та звітність;
- Open/Closed Principle (OCP): архітектура відкрита для розширення та закрита для модифікації. Створення нового способу сповіщення (наприклад, запис у журнал чи відправка в мережу) реалізується новим класом і не вимагає внесення жодних змін у код StudentRegistry;
- Dependency Inversion Principle (DIP): модуль високого рівня StudentRegistry залежить від абстракцій Repository[Student] (ABC) та Notifier (Protocol), а не від їхніх низькорівневих конкретних реалізацій.

```python
class StudentRegistry:
    # реєстр студентів із контролем цілісності та сповіщеннями
    def __init__(self, repository: Repository[Student], notifier: Notifier) -> None:
        self._repository = repository
        self._notifier = notifier

    def add(self, student_id: int, student: Student) -> Student:
        # валідуєм id і додаєм копію студента з узгодженим id
        if (
            isinstance(student_id, bool)
            or not isinstance(student_id, int)
            or student_id <= 0
        ):
            raise ValueError("student_id must be a positive integer")

        existing_id = student.student_id
        if existing_id is not None and existing_id != student_id:
            raise ValueError("Inconsistent student_id between entity and registry key")

        stored_student = replace(student, student_id=student_id)

        self._repository.add(student_id, stored_student)
        self._notifier.send(f"Added student {student_id}: {stored_student.full_name}")
        return stored_student
```

10. Провів два порівняльні архітектурні експерименти для вивчення характеристик дизайну.

Експеримент 1 (Hardcoded vs Injected Notifier): порівняв сервіс із жорстко зашитою залежністю ConsoleNotifier та сервіс із впровадженням через конструктор:

```python
# --- Експеримент 1: Внутрішня залежність проти Dependency Injection ---


class HardcodedNotifierService:
    # варіант a: сервіс створює конкретну залежність всередині
    def __init__(self) -> None:
        self._notifier: Notifier = ConsoleNotifier()

    def register_student(self, student_id: int, full_name: str) -> str:
        # реєстрація з жорстко прив'язаним сповіщувачем
        message = f"Audit: student {student_id} ({full_name}) registered"
        self._notifier.send(message)
        return message


class InjectedNotifierService:
    # варіант b: сервіс отримує абстракцію через конструктор (DIP)
    def __init__(self, notifier: Notifier) -> None:
        self._notifier = notifier

    def register_student(self, student_id: int, full_name: str) -> str:
        # реєстрація з інжектованим сповіщувачем
        message = f"Audit: student {student_id} ({full_name}) registered"
        self._notifier.send(message)
        return message
```

Результати аналізу експерименту 1 зведено в порівняльну таблицю:

Критерій порівняння	**Hardcoded Notifier (Варіант A)**	**Injected Notifier (Варіант B, DIP)**
Зв'язність (Coupling)	Висока: пряма залежність від конкретного класу ConsoleNotifier	Низька: слабка зв'язність через абстрактний протокол Notifier
Тестованість (Testability)	Низька: неможливо перевірити відправку без перехоплення stdout	Висока: легко підставити RecordingNotifier для аудиту й тестів

Замінність реалізації	Складна: зміна каналу вимагає редагування вихідного коду сервісу	Гнучка (OCP): нова реалізація передається ззовні без змін коду
Відповідність DIP	Порушує DIP: модуль високого рівня залежить від конкретики	Відповідає DIP: високий і низький рівні залежать від абстракції

Експеримент 2 (Inheritance vs Composition): реалізував форматування картки студента двома способами: через шаблонний метод у базовому класі BaseStudentCardFormatter (наслідування) та через конвеєр незалежних функцій-кроків CompositeStudentCardProcessor (композиція):

```python
class BaseStudentCardFormatter(ABC):
    # варіант a: базовий клас із шаблоном обробки (наслідування)
    def format_record(self, raw: str) -> str:
        parsed = self.parse(raw)
        validated = self.validate(parsed)
        return self.render(validated)

    def parse(self, raw: str) -> dict[str, str]:
        return default_parse(raw)

    def validate(self, data: dict[str, str]) -> dict[str, str]:
        return default_validate(data)

    @abstractmethod
    def render(self, data: dict[str, str]) -> str: ...


class TemplateStudentCardFormatter(BaseStudentCardFormatter):
    # конкретна реалізація рендерингу в ієрархії наслідування
    def render(self, data: dict[str, str]) -> str:
        return default_render(data)


class CompositeStudentCardProcessor:
    # варіант b: композиція з незалежних функцій-кроків
    def __init__(
        self,
        parser: Callable[[str], dict[str, str]],
        validator: Callable[[dict[str, str]], dict[str, str]],
        renderer: Callable[[dict[str, str]], str],
    ) -> None:
        self._parser = parser
        self._validator = validator
        self._renderer = renderer

    def process(self, raw: str) -> str:
        # послідовне застосування переданих компонентів
        return self._renderer(self._validator(self._parser(raw)))
```

Результати порівняння експерименту 2 наведено в таблиці:

Критерій порівняння	**Inheritance Formatter (Варіант A)**	**Composition Processor (Варіант B)**
Зв'язність (Coupling)	Висока: дочірній клас міцно прив'язаний до структури базового	Мінімальна: незалежні кроки об'єднуються через інтерфейс Callable
Розширюваність (Extensibility)	Обмежена: кожна комбінація кроків потребує нового підкласу	Динамічна: функції парсингу, валідації та рендерингу легко замінювати

Тестованість (Testability)	Середня: тестування вимагає інстанціювання всього класу	Відмінна: парсер, валідатор і рендерер тестуються повністю окремо
Повторне використання (Reuse)	Низьке: методи парсингу зашиті в ієрархію форматування	Високе: окремі функції можна перевикористовувати в інших модулях

11. Для забезпечення максимальної повноти та якості реалізації виконав чотири додаткові можливості з переліку завдань:
1) Parameterized generic repository: узагальнений базовий репозиторій Repository[T] та InMemoryRepository[T], що безпечно типізуються будь-яким доменним типом;
2) Runtime-checkable Protocol: протокол Notifier із підтримкою dynamic type checking через isinstance під час виконання програми;
3) Custom collection class: реєстр StudentRegistry надає повноцінний інтерфейс колекції завдяки методам __len__, __iter__, __contains__ та __getitem__;
4) Immutable value objects: класи Group, Grade та AcademicRecord є повністю незмінними за рахунок frozen=True, що гарантує захист від небажаних мутацій.

12. Реалізував наскрізний демонстраційний сценарій у модулі lab4_demo.py, який покроково показує роботу всієї системи: створення реєстру через DI, додавання студентів, захист від некоректного редагування, пошук за прізвищем та групою, розрахунок аналітики, додавання оцінок до AcademicRecord, заміну нотифікатора на RecordingNotifier, колекційні dunder-методи, звітність через ReportingService та запуск архітектурних експериментів:

```
def run_demo() -> None:
    # 1. ініціалізація компонентів через dependency injection
    print("--- 1. Ініціалізація репозиторію та реєстру ---")
    repo = InMemoryRepository[Student]()
    console_notifier = ConsoleNotifier()
    registry = StudentRegistry(repository=repo, notifier=console_notifier)

    # 2. додавання студентів
    print("\n--- 2. Додавання студентів із числовими ID ---")
    s1 = Student("Олена", "Коваль", "ФеП-33", 91.5)
    s2 = Student("Іван", "Франко", "ФеП-33", 88.0)
    s3 = Student("Тарас", "Шевченко", "ФеП-34", 96.5)
    s4 = Student("Леся", "Українка", "ФеП-34", 94.0)

    registry.add(1, s1)
    registry.add(2, s2)
    registry.add(3, s3)
    registry.add(4, s4)

    # 3. редагування та атомарний захист
    print("\n--- 3. Редагування та атомарна валідація ---")
    updated = registry.edit(2, average_grade=89.5)
    print(
        f"Успішно оновлено студента 2: {updated.full_name}, новий бал: {updated.average_grade}"
    )

    # перевірка атомарності при некоректних даних
    try:
        registry.edit(2, average_grade=150.0)
    except ValueError as err:
        print(f"Спроба некоректного редагування відхилена: {err}")
    current_s2 = registry.get_by_id(2)
    print(
        f"Стан студента 2 після помилки: бал {current_s2.average_grade if current_s2 else 'None'} (не змінився)"
    )
```

```
    # 4. пошук за прізвищем та групою
    print("\n--- 4. Пошук за прізвищем та групою ---")
    kowals = registry.search_by_last_name("  коваль ")
    print(f"Знайдено за прізвищем 'коваль': {len(kowals)} студент(ів)")

    fep33_group = Group("ФеП-33")
    group_students = registry.search_by_group(fep33_group)
    print(f"Студенти групи {fep33_group}: {[s.full_name for s in group_students]}")

    # 5. середній бал та найкращий студент
    print("\n--- 5. Аналітика реєстру ---")
    avg_total = registry.get_average_grade()
    avg_fep33 = registry.get_average_grade(fep33_group)
    best_student = registry.find_best_student()
    best_fep33 = registry.find_best_student(fep33_group)

    print(
        f"Загальний середній бал: {avg_total:.2f}"
        if avg_total is not None
        else "Немає даних"
    )
    print(
        f"Середній бал групи ФеП-33: {avg_fep33:.2f}"
        if avg_fep33 is not None
        else "Немає даних"
    )
    print(
        f"Найкращий студент загалом: {best_student.full_name} ({best_student.average_grade})"
        if best_student
        else "Немає"
    )
    print(
        f"Найкращий студент ФеП-33: {best_fep33.full_name} ({best_fep33.average_grade})"
        if best_fep33
        else "Немає"
    )
```

```
    # 6. робота з Grade та AcademicRecord
    print("\n--- 6. Доменні об'єкти Grade та AcademicRecord ---")
    g1 = Grade(92.0)
    g2 = Grade(95.5)
    g3 = Grade(88.0)
    record = AcademicRecord(student_id=1, grades=(g1, g2))
    print(
        f"Академічна картка студента 1: оцінок {len(record)}, середній бал: {record.average:.2f}"
        if record.average
        else ""
    )

    record_extended = record.add_grade(g3)
    print(
        f"Після додавання оцінки {float(g3)}: оцінок {len(record_extended)}, середній: {record_extended.average:.2f}"
        if record_extended.average
        else ""
    )

    # 7. поліморфізм та заміна Notifier
    print("\n--- 7. Заміна сповіщувача через Dependency Injection ---")
    audit_recorder = RecordingNotifier()
    audit_registry = StudentRegistry(repository=repo, notifier=audit_recorder)
    audit_registry.edit(1, average_grade=93.0)
    print(f"Записано сповіщень аудиту: {len(audit_recorder.messages)}")
    for msg in audit_recorder.messages:
        print(f"  [audit log]: {msg}")

    # 8. колекційні dunder-методи реєстру
    print("\n--- 8. Колекційні dunder-методи реєстру ---")
    print(f"len(registry): {len(registry)}")
    print(f"1 in registry: {1 in registry}")
    print(f"99 in registry: {99 in registry}")
    print(f"registry[1]: {registry[1].full_name}")
```

```
    # 9. звітність через ReportingService та TypedDict
    print("\n--- 9. Генерація типізованих звітів (ReportingService) ---")
    reporter = ReportingService(registry)
    summary = reporter.get_summary()
    print("Підсумок реєстру (RegistrySummary):")
    for key, val in summary.items():
        print(f"  {key}: {val}")

    payloads = reporter.export_students()
    print(f"\nЕкспортовано записів TypedDict: {len(payloads)}")
    for p in payloads[:2]:
        print(f"  {p}")

    # 10. результати експериментів з дизайну
    print("\n--- 10. Порівняльні архітектурні експерименти ---")
    exp_results = run_design_experiments()
    print(
        f"Експеримент 1 (Hardcoded vs Injected Notifier) - вивід ідентичний: {exp_results['notifier_match']}"
    )
    print(f"  Зразок повідомлення: {exp_results['notifier_output']}")
    print(
        f"Експеримент 2 (Inheritance vs Composition) - вивід ідентичний: {exp_results['formatter_match']}"
    )
    print(f"  Зразок рядка: {exp_results['formatter_output']}")


if __name__ == "__main__":
    run_demo()
```

Запустив модуль lab4_demo.py через віртуальне середовище. Нижче наведено реальний консольний вивід виконання сценарію, розділений на логічні частини:

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m student_manager.lab4_demo
--- 1. Ініціалізація репозиторію та реєстру ---

--- 2. Додавання студентів із числовими ID ---
Added student 1: Олена Коваль
Added student 2: Іван Франко
Added student 3: Тарас Шевченко
Added student 4: Леся Українка

--- 3. Редагування та атомарна валідація ---
Updated student 2: Іван Франко
Успішно оновлено студента 2: Іван Франко, новий бал: 89.5
Спроба некоректного редагування відхилена: Average grade must be between 0 and 100
Стан студента 2 після помилки: бал 89.5 (не змінився)

--- 4. Пошук за прізвищем та групою ---
Знайдено за прізвищем 'коваль': 1 студент(ів)
Студенти групи ФеП-33: ['Олена Коваль', 'Іван Франко']
```

```
--- 5. Аналітика реєстру ---
Загальний середній бал: 92.88
Середній бал групи ФеП-33: 90.50
Найкращий студент загалом: Тарас Шевченко (96.5)
Найкращий студент ФеП-33: Олена Коваль (91.5)

--- 6. Доменні об'єкти Grade та AcademicRecord ---
Академічна картка студента 1: оцінок 2, середній бал: 93.75
Після додавання оцінки 88.0: оцінок 3, середній: 91.83

--- 7. Заміна сповіщувача через Dependency Injection ---
Записано сповіщень аудиту: 1
  [audit log]: Updated student 1: Олена Коваль

--- 8. Колекційні dunder-методи реєстру ---
len(registry): 4
1 in registry: True
99 in registry: False
registry[1]: Олена Коваль
```

```
--- 9. Генерація типізованих звітів (ReportingService) ---
Підсумок реєстру (RegistrySummary):
  total_students: 4
  overall_average: 93.25
  best_student_id: 3
  best_student_name: Тарас Шевченко

Експортовано записів TypedDict: 4
  {'student_id': 1, 'first_name': 'Олена', 'last_name': 'Коваль', 'group': 'ФеП-33', 'average_grade': 93.0}
  {'student_id': 2, 'first_name': 'Іван', 'last_name': 'Франко', 'group': 'ФеП-33', 'average_grade': 89.5}

--- 10. Порівняльні архітектурні експерименти ---
Audit: student 1 (Олена Коваль) registered
Експеримент 1 (Hardcoded vs Injected Notifier) - вивід ідентичний: True
  Зразок повідомлення: Audit: student 1 (Олена Коваль) registered
Експеримент 2 (Inheritance vs Composition) - вивід ідентичний: True
  Зразок рядка: STUDENT#10: Ірина Бондар [ФеП-33] - 95.5 балів
```

13. Для гарантії надійності розробив набір цільових модульних тестів (tests/test_domain.py, tests/test_registry.py та tests/test_registry_boundaries.py), які

перевіряють граничні значення оцінок, незмінність Value Objects, обчислення середнього бала в картці, відхилення дублікатів ключів у репозиторії, атомарність редагування та поліморфізм протоколу Notifier. Фрагменти тестів наведено нижче:

```
    def test_grade_valid_values(self) -> None:
        from student_manager.domain import Grade

        grade = Grade(95.5)
        self.assertEqual(float(grade), 95.5)
        self.assertEqual(str(grade), "95.5")

        boundary_low = Grade(0.0)
        self.assertEqual(float(boundary_low), 0.0)

        boundary_high = Grade(100.0)
        self.assertEqual(float(boundary_high), 100.0)

    def test_grade_rejects_out_of_bounds_and_non_finite(self) -> None:
        from student_manager.domain import Grade

        invalid_values = [
            -0.1,
            100.1,
            -50.0,
            150.0,
            float("nan"),
            float("inf"),
            float("-inf"),
            math.nan,
        ]
        for val in invalid_values:
            with self.subTest(val=val), self.assertRaises(ValueError):
                Grade(val)

    def test_grade_rejects_bool(self) -> None:
        from student_manager.domain import Grade

        with self.assertRaises((TypeError, ValueError)):
            Grade(True)  # type: ignore[arg-type]
```

```python
    def test_in_memory_repository_crud_flow(self) -> None:
        from student_manager.models import Student
        from student_manager.repositories import InMemoryRepository

        repo: InMemoryRepository[Student] = InMemoryRepository()
        student1 = Student("Олена", "Коваль", "ФеП-33", 90.0)
        repo.add(1, student1)

        self.assertEqual(repo.get(1), student1)
        self.assertIsNone(repo.get(999))
        self.assertEqual(repo.all(), (student1,))
        self.assertEqual(len(repo), 1)
        self.assertIn(1, repo)
        self.assertNotIn(2, repo)

        student1_updated = Student("Олена", "Коваль", "ФеП-33", 95.0)
        repo.update(1, student1_updated)
        self.assertEqual(repo.get(1), student1_updated)

    def test_in_memory_repository_rejects_duplicate_key(self) -> None:
        from student_manager.models import Student
        from student_manager.repositories import InMemoryRepository

        repo: InMemoryRepository[Student] = InMemoryRepository()
        s1 = Student("Олена", "Коваль", "ФеП-33", 90.0)
        s2 = Student("Іван", "Франко", "ФеП-33", 92.0)
        repo.add(1, s1)
        with self.assertRaises(ValueError):
            repo.add(1, s2)
```

```python
import unittest

from student_manager.models import Student
from student_manager.registry import RecordingNotifier, StudentRegistry
from student_manager.repositories import InMemoryRepository


class RegistryBoundaryTests(unittest.TestCase):
    def setUp(self):
        self.registry = StudentRegistry(InMemoryRepository(), RecordingNotifier())
        self.registry.add(1, Student("Anna", "Koval", "A", 90))

    def test_bool_id_does_not_alias_student_one(self):
        with self.assertRaises((TypeError, ValueError)):
            self.registry.get_by_id(True)
        with self.assertRaises((TypeError, ValueError)):
            self.registry.edit(True, average_grade=0)
        self.assertEqual(self.registry[1].average_grade, 90)

    def test_export_preserves_grade_precision(self):
        from student_manager.registry import ReportingService

        self.registry.edit(1, average_grade=90.125)
        self.assertEqual(
            ReportingService(self.registry).export_students()[0]["average_grade"],
            90.125,
        )
```

Виконав повний запуск цільових тестів у терміналі PowerShell:

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m unittest tests/test_domain.py
tests/test_registry.py tests/test_registry_boundaries.py
Audit: student 1 (Олена Коваль) registered
Audit: student 1 (Олена Коваль) registered
.....................................
----------------------------------------------------------------------
Ran 38 tests in 0.010s

OK
```

14. Виконав обов'язковий експеримент зі статичної перевірки типів за допомогою mypy. Для цього у виділеному експериментальному модулі навмисно змоделював три типові помилки типізації:
1) змінній цілого типу присвоєно рядок (student_count: int = '10');
2) функція повертає рядок замість очікуваного цілого числа (def get_default_grade_points() -> int: return '100');
3) у репозиторій, типізований для екземплярів Student (InMemoryRepository[Student]), передається об'єкт типу Group.
Вихідний код модуля з помилками:

```python
from student_manager.domain import Group
from student_manager.models import Student
from student_manager.repositories import InMemoryRepository

# помилка 1: змінній типу int присвоєно str
student_count: int = "10"


# помилка 2: функція повертає str замість int
def get_default_grade_points() -> int:
    return "100"


# помилка 3: у репозиторій InMemoryRepository[Student] передається екземпляр Group
def register_sample_entity() -> None:
    repo: InMemoryRepository[Student] = InMemoryRepository()
    group: Group = Group("ФеП-33")
    repo.add(1, group)
```

Запустив статичний аналізатор mypy з прапорцем --follow-imports=silent та передачею MYPYPATH=src. Аналізатор точно виявив усі три порушення контракту типів:

```
PS F:\university\3yr-1\professional_python> $env:MYPYPATH='src'; uv run --with mypy mypy --follow-imports=silent .local_workflow/lab4/type_experiment_invalid.py
.local_workflow\lab4\type_experiment_invalid.py:6: error: Incompatible types in assignment (expression has type "str", variable has type "int")  [assignment]
.local_workflow\lab4\type_experiment_invalid.py:11: error: Incompatible return value type (got "str", expected "int")  [return-value]
.local_workflow\lab4\type_experiment_invalid.py:18: error: Argument 2 to "add" of "InMemoryRepository" has incompatible type "Group"; expected "Student"  [arg-type]
Found 3 errors in 1 file (checked 1 source file)
```

Після фіксації діагностики виправив усі три помилки у файлі type_experiment.py: призначив змінній коректне число 10, повернув ціле число 100 та передав у репозиторій валідний екземпляр Student:

```python
from student_manager.models import Student
from student_manager.repositories import InMemoryRepository

# виправлено 1: присвоєно значення типу int
student_count: int = 10


# виправлено 2: функція повертає значення типу int
def get_default_grade_points() -> int:
    return 100


# виправлено 3: у репозиторій передається правильний екземпляр Student
def register_sample_entity() -> None:
    repo: InMemoryRepository[Student] = InMemoryRepository()
    student: Student = Student("Олена", "Коваль", "ФеП-33", 95.0)
    repo.add(1, student)
```

Повторний запуск mypy підтвердив повну типобезпеку та відсутність помилок:

```
PS F:\university\3yr-1\professional_python> $env:MYPYPATH='src'; uv run --with mypy mypy --follow-imports=silent .local_workflow/lab4/type_experiment.py
Success: no issues found in 1 source file
```

Зазначу, що статичний аналіз виконано цільовим чином для доменної моделі та експериментального модуля; глобальна конфігурація проєкту для всіх модулів формується на рівні репозиторію, тому цільова ізольована перевірка забезпечила точну та достовірну фіксацію результатів.

Висновок:

Під час виконання лабораторної роботи опанував проектування професійної об'єктно-орієнтованої архітектури в Python та застосування сучасної системи статичної типізації. Побудував узгоджену доменну модель системи обліку студентів із використанням dataclass, незмінних Value Objects, властивостей @property, узагальнених репозиторіїв Repository[T] та протоколів structural subtyping Notifier. Переконався у значних перевагах композиції та інверсії залежностей (DIP) перед жорстким наслідуванням, що суттєво підвищує тестованість і гнучкість коду. Реалізував ключові dunder-методи для підтримки поведінки стандартних колекцій, провів порівняльні архітектурні експерименти, підтвердив надійність системи 38 автоматичними тестами та дослідив роботу статичного аналізатора mypy, успішно виявивши та виправивши навмисно внесені помилки типів.